

A faint, stylized line drawing of a person with glasses holding a large book. The background features a large semi-circle and several small diamonds.

Estrutura gramatical para PLN

Você vai aprender os aspectos teóricos e práticos de estruturas gramaticais e formalismos relacionados ao processamento de linguagem natural. Essa abordagem é muito interessante, pois vai fazer você lembrar das aulas de português do primeiro e segundo grau. No entanto, agora, utilizamos recursos da inteligência artificial para generalizar as soluções, o que é fundamental para um profissional de informática.

Prof. Sérgio Assunção Monteiro

Preparação

É importante usar o Google Colab para desenvolver os exemplos na linguagem Python – ele é gratuito. Tudo que você vai precisar é de uma conta no Gmail e, claro, praticar os exemplos e exercícios que propusermos. Em especial, vamos utilizar as funcionalidades da biblioteca NLTK. Os códigos da parte prática estão neste [link](#).

Objetivos

- Descrever estruturas gramaticais no contexto do processamento de linguagem natural.
- Identificar a biblioteca NLTK aplicada ao processamento de linguagem natural.
- Implementar técnicas de processamento de linguagem natural com Python.

Introdução

Atualmente, podemos constatar o grande crescimento das aplicações de processamento de linguagem natural (PLN). Essas aplicações abrangem desde o atendimento automático de clientes até interações mais sofisticadas com ferramentas de buscas. O fato é que a PLN tem se popularizado devido ao grande potencial de reduzir custos e aumentar a precisão das respostas.

Vamos conhecer as estruturas gramaticais e os formalismos, que é uma das abordagens de PLN que explora elementos léxicos, sintáticos e semânticos. Além disso, vamos explorar a biblioteca NLTK do Python que já nos disponibiliza diversas funcionalidades.

Visão geral sobre estruturas gramaticais

Confira, neste vídeo, conceitos essenciais sobre estruturas gramaticais e sua relação com o processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceitos sobre Processamento de Linguagem Natural

A capacidade de poder expressar nossos pensamentos através de palavras nos permite realizar diversas ações. Para que possamos melhorar a qualidade dessas ações, precisamos utilizar uma comunicação efetiva no sentido de facilitar o entendimento das nossas ideias. Uma das formas mais comuns de comunicação ocorre por meio de palavras que devem ser combinadas a partir de uma estrutura lógica.



Formas de comunicação.

Essa estrutura lógica pode ser feita através do texto ou da fala. Então, quando pensamos em uma forma de desenvolver uma solução com o uso de sistemas computacionais, precisamos compreender essas estruturas lógicas. Uma das áreas que trabalha exatamente com esses aspectos que abordamos é conhecida como **processamento de linguagem natural (PLN)**.

A PLN é uma das áreas da inteligência artificial (IA) que tem como objetivo mapear as regras das estruturas lógicas da linguagem humana e, através de diversas técnicas, aplicar soluções automatizadas que sejam capazes de extrair informações de textos ou falas. Ao realizar esse processo de extrair informações, podemos aplicar outras soluções

que nos permitem realizar diversas tarefas, como:



Analisar opiniões de usuários



Gerar dados relevantes para realizar estudos estatísticos



Usar palavras-chave como comandos

Existem muitas outras aplicações, mas com esses exemplos, já podemos ter uma dimensão dos benefícios que podemos obter e dos desafios que precisamos superar, para que os sistemas computacionais possam entender e interpretar a linguagem humana.

A PLN é uma área multidisciplinar que abrange as seguintes áreas: inteligência artificial, linguística computacional, interação humano-computador etc. Na sequência, vamos estudar alguns aspectos essenciais sobre gramáticas e formalismos.

Conceitos sobre estruturas gramaticais e formalismos

Quando falamos em estruturas gramaticais e formalismos, precisamos primeiro entender o que é a **sintaxe**.

A sintaxe é um conjunto de regras que trata como as palavras podem ser estruturadas entre si. Essas regras estão associadas a um contexto que abrange as linguagens naturais, como também as linguagens de programação.

A seguir, vamos conferir cada um desses contextos.

Estruturas gramaticais no contexto das linguagens naturais

Em nosso idioma, uma oração é composta por sujeito e predicado. No estudo do sujeito, podemos ter:

1 Sujeito simples

Ex.: O cachorro é esperto.

Formado por um único núcleo.

2

Sujeito composto

Ex.: O cachorro e o gato podem conviver em harmonia.

Composto por um ou mais núcleos.

3

Sujeito oculto

Ex.: Vamos estudar PLN.

Ocorre quando o sujeito da frase fica subentendido.

4

Sujeito indeterminado

Ex.: Estão estudando sobre PLN.

Ocorre quando não é possível determinar o sujeito da oração.

5

Sujeito inexistente

Ex.: Choveu o dia todo.

Ocorre quando utilizamos verbos impessoais.

Agora, em relação ao predicado, podemos ter:

Predicado verbal

Ex.: Nós estudamos PLN.

Ocorre quando a oração usa um verbo que expressa ação, estado ou fenômeno, como núcleo.

Predicado nominal

Ex.: O cachorro está feliz.

Ocorre quando usamos um verbo de ligação como núcleo da oração.

Predicado verbo-nominal

Ex.: Estudamos energicamente sobre técnicas de PLN.

É uma combinação dos dois casos anteriores.

As regras gramaticais nos permitem utilizar as linguagens naturais de um modo correto e formal. No entanto, sabemos que a nossa comunicação no dia a dia é mais flexível e aceita gírias e expressões coloquiais. A PLN também pode ser aplicada nesse tipo de cenário.

Estruturas gramaticais no contexto das linguagens de programação

Agora, vamos abordar a PLN no contexto das linguagens de programação. Nesse caso, a sintaxe é definida no sentido que cada linguagem de programação utiliza estruturas rígidas para escrever instruções ou declarações que o computador possa entender e executar.

Cada linguagem de programação especifica a ordem e a disposição de palavras-chave, operadores, sinais de pontuação e outros elementos para que seja criado um código válido.

É interessante analisarmos como existem diversas linguagens de programação, vide C, C++, C#, Java, JavaScript e Python. Cada uma delas possui características particulares que as tornam mais adequadas para desenvolvermos soluções para determinados tipos de problemas. No entanto, todas elas possuem gramáticas que definem de forma rígida como os elementos que a compõem podem ser combinados para gerarmos programas corretos.

Atividade 1

Atualmente, temos diversas tecnologias para capturar dados à nossa disposição. No entanto, ainda é comum que muitos desses dados sejam obtidos de forma não estruturada. Em muitas situações, a solução mais adequada para tratar esses problemas é através das técnicas de processamento de linguagem natural (PLN). Nesse sentido, selecione a opção correta que apresenta uma situação adequada para aplicar PLN.

A

Cadastro de dados pessoais.

B

Registro de dados bancários.

C

Análise de dados estatístico.

D

Análise de comentários em um blog.

E

Reconhecimento de imagens.



A alternativa D está correta.

A PLN é adequada para processar textos ou falas, por exemplo, os comentários dos usuários em um blog. Normalmente, esse tipo de aplicação trabalha com dados não estruturados. No entanto, existem muitas situações em que trabalhamos com dados não estruturados, como no processamento de imagens, caso em que a PLN não é adequada.

Principais estruturas gramaticais

Confira, nesse vídeo, algumas das principais gramáticas, com destaque para as características importantes que precisamos compreender antes de avançarmos no nosso estudo de processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Importância das estruturas gramaticais para PLN

Sabe-se que as gramáticas ocupam um lugar essencial para trabalharmos com PLN, pois, é por meio dessas estruturas que podemos capturar os relacionamentos entre diferentes elementos linguísticos – chamados de classes gramaticais –, tais como: substantivos, verbos, adjetivos e as regras que tratam de sua disposição.

As estruturas gramaticais são um conjunto de regras e princípios que estabelecem como podemos realizar a organização e combinação das palavras, orações, frases e sentenças para formar expressões linguísticas que tenham sentido.

Os formalismos, por sua vez, são estruturas ou notações específicas usadas para representar e formalizar estruturas gramaticais. São através deles que podemos descrever a gramática de uma linguagem, muitas vezes, usando símbolos, regras e restrições.

Principais gramáticas para PLN

Existem várias estruturas gramaticais que podemos aplicar para PLN. O objetivo geral de todas elas é formalizar as regras para composição de palavras, servindo para reconhecer padrões e gerar linguagem. A seguir, vamos conhecer melhor algumas dessas estruturas gramaticais.

Gramática livre de contexto

É um formalismo muito utilizado para descrever as regras sintáticas das linguagens naturais. Trata-se de uma gramática composta por um conjunto de símbolos não terminais, um conjunto de símbolos terminais, outro conjunto de regras de produção que definem como fazemos para gerar sentenças válidas em uma linguagem e uma variável inicial.

Gramática regular

É um caso particular das gramáticas livres de contexto. As gramáticas regulares têm como objetivo descrever as chamadas linguagens regulares. Elas são muito úteis para representar expressões regulares e autômatos de estado finito.

Gramática de dependência

Tem como foco descrever as relações entre as palavras de uma frase. As sentenças são representadas como grafos dirigidos. Um grafo dirigido G é definido matematicamente, como:

$$G = (V, A)$$

Em que:

- V : representa o conjunto de nós.
- A : representa o conjunto de arestas.

No caso da gramática de dependências, as palavras são os nós e as relações gramaticais são representadas pelas arestas. Além disso, cada palavra depende de uma ou mais palavras na frase.

Gramática de estrutura frasal

Também é conhecida como gramática constituinte. Trata-se de um formalismo gramatical que define a estrutura de frases. Ela usa estruturas hierárquicas para representar os elementos constituintes de sentenças, como frases nominais, frases verbais etc.

Aqui, o nosso objetivo era ter uma visão geral sobre algumas das principais gramáticas aplicadas para PLN. Portanto, agora é hora de fazer o exercício para consolidar o que acabamos de estudar. Vamos lá!

Atividade 2

Para representarmos uma linguagem, precisamos de uma ferramenta que nos ajude a formalizá-la, e a ferramenta que nos ajuda nesse processo é a gramática. No entanto, sabemos que existem diversos tipos de gramáticas. Nesse sentido, selecione a opção correta que justifica a necessidade da existência de tantas gramáticas.

A

Modelar a estrutura das sentenças.

B

Eliminar as ambiguidades de uma linguagem.

C

Aumentar a capacidade de flexibilizar uma linguagem.

D

Representar cada linguagem por apenas um único tipo de gramática.

E

Criar uma equivalência de representação com expressões regulares.



A alternativa A está correta.

Cada linguagem tem as próprias particularidades. Portanto, precisamos de diferentes tipos de gramáticas para representar essas particularidades. Isso significa que uma mesma linguagem pode ser representada por mais de um tipo de gramática dependendo do que queremos destacar.

GLC aplicada para PLN

Confira, neste vídeo, aspectos conceituais e um exemplo da gramática livre de contexto (GLC). A GLC é o ponto inicial para entendermos, na prática, como podemos formalizar as regras sintáticas de uma linguagem e aplicar algoritmos para processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais da gramática livre de contexto

Uma gramática livre de contexto (GLC) é uma forma de representarmos estruturas sintáticas de uma linguagem. Ela é composta por quatro componentes:

1

Conjunto de não terminais
Símbolo: V

Os não terminais são variáveis sintáticas que utilizamos para derivar palavras.

2

Conjunto de terminais
Símbolo: T

É conhecido como conjunto de tokens. As palavras são formadas através dos símbolos dos terminais.

3

Conjunto de produções
Símbolo: P

É composto por regras sintáticas para formação dos termos reconhecidos pela gramática através da combinação dos símbolos terminais e não terminais.

4 Símbolo inicial

Símbolo: S

A gramática precisa iniciar a partir de um símbolo não terminal. Normalmente, utilizamos o símbolo S para representar a variável de produção inicial da gramática.

A seguir, vamos analisar a GLC de um modo formal e aplicar um exemplo.

Formalização matemática da gramática livre de contexto

Matematicamente, podemos descrever uma gramática livre de contexto da seguinte forma: uma gramática G pode ser escrita como uma 4-tupla (V, T, S, P) , em que:

- V = conjunto de símbolos não terminais, ou variáveis.
- T = conjunto de símbolos terminais.
- S = Símbolo inicial onde $S \in V$.
- P = Regras de produção para terminais e não terminais. Tem a forma $\alpha \rightarrow \beta$, onde α e β são strings em $V \cup T$ e, pelo menos, um símbolo de α pertence a V .

Vejamos um exemplo de gramática livre de contexto!

Seja $G = (V, T, S, P)$, sendo que:

- $V = \{S\}$
- $T = \{a, b\}$
- $P = \{S \rightarrow abS \mid a\}$

Agora, precisamos entender o que essa gramática faz. Ela é capaz de gerar palavras de uma linguagem formada por símbolos "a" e "b" com uma combinação muito específica. No caso, as palavras geradas pela nossa gramática são dadas por:

$$L(G) = \{aba, ababa, abababa, \dots\}$$

Isso ocorre devido às regras de produção. Por exemplo, vamos considerar a palavra "ababa". Ela é reconhecida pela nossa gramática através de uma sequência de passos. Observe!

1

Passo 1

Devemos associar a palavra de entrada "ababa" com a variável inicial S .

2

Passo 2

A regra de produção de S que se encaixa com a palavra de entrada é "abs", sendo que o conteúdo associado à variável S , agora, é dado por "aba".

3 Passo 3

Vamos encontrar a regra de produção associada à "aba". No caso, novamente, temos que a regra que se encaixa é "abs". Nesse momento, a variável S está associada ao conteúdo "a".

4

Passo 4

Precisamos encontrar a regra associada ao conteúdo "a". No caso, simplesmente usamos a regra de produção " $S \rightarrow a$ ".

Assim, concluímos o processo de reconhecimento, o chamado **processo de derivação**.

Árvore de derivação

Uma forma bastante útil de realizarmos o processo de reconhecimento de estruturas sintáticas nas gramáticas livre de contexto é através da árvore de derivação. Vamos aplicar uma árvore de derivação para o exemplo que vimos anteriormente.

No início, a nossa gramática recebeu a palavra "ababa". Isso significa que precisamos associá-la à variável inicial da gramática que, no nosso caso, é S . Confira esse passo a passo nas imagens a seguir.

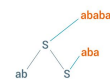
Passo 1

Como falamos, a palavra "ababa" é associada à variável inicial da gramática que, no nosso caso, é S .



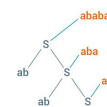
Passo 2

A regra de produção associada a essa palavra inicial é " $S \rightarrow abS$ ". Veja o resultado na árvore de derivação!



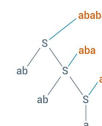
Passo 3

Consiste em encontrarmos a próxima regra de produção, que é dada, novamente, por " $S \rightarrow abS$ ". Observe o resultado na árvore!



Passo 4

Temos que encontrar a regra que deriva "a", que, no caso, é dada por " $S \rightarrow a$ ". Mais uma vez, atenção à árvore de derivação.



Percebeu como a árvore de derivação é uma ferramenta útil para visualizarmos o reconhecimento de estruturas sintáticas? Agora, vamos consolidar nossos conhecimentos de forma prática.

Atividade 3

A formalização da linguagem é uma forma muito eficiente de transmissão e compreensão de ideias. Há situações em que essa comunicação é bastante precisa e, assim, podemos utilizar gramáticas livres de

contexto (GLC) para representar estruturas sintáticas e validar textos de entrada. Nesse sentido, selecione a opção que apresenta corretamente um exemplo prático em que podemos aplicar a GLC.

A

Para verificar se as redações de estudantes estão corretas.

B

Para validar as regras da língua portuguesa em letras de música.

C

Na verificação das informações de destinatários em correspondências.

D

Na verificação dos significados de termos coloquiais.

E

Para corrigir textos em comentários dos usuários sobre um determinado serviço.



A alternativa C está correta.

A GLC é um modelo muito eficiente para representar e verificar regras sintáticas. No entanto, ela possui uma estrutura rígida que não permite que se adapte dinamicamente às flexibilidades da língua coloquial. Apesar desse tipo de limitação, ela pode ser usada em diversas situações práticas, por exemplo, para verificar as informações de destinatários em correspondências, pois é uma situação em que ocorre uma estrutura rígida, ou seja, toda correspondência deve conter o nome do destinatário e informações sobre o endereço.

Gramática de estrutura frasal

Confira, neste vídeo, os conceitos e alguns exemplos da gramática estrutural frasal. Essa gramática possui elementos mais elaborados que nos permitem fazer representações mais robustas de uma linguagem para poder aplicar algoritmos para processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Fundamentos da gramática de estrutura frasal

A gramática de estrutura frasal também é conhecida como gramática de constituintes. Nós a utilizamos para apresentar regras e princípios relacionados à organização e à combinação das palavras dentro de uma oração para formar frases gramaticalmente corretas e significativas. Nesse sentido, ela abrange aspectos que tratam de ordem das palavras, estrutura da frase, relações sintáticas e papéis dos diferentes componentes dentro da frase.

Quando estudamos a língua portuguesa, aprendemos a analisar as frases através de suas partes constituintes, que são os substantivos, verbos, adjetivos, advérbios, as preposições e conjunções. As regras gramaticais servem para descrever como essas partes podem ser organizadas na formação de frases significativas através das regras para concordância sujeito-verbo, tempo verbal, tipos de sentença (declarativa, interrogativa, imperativa etc.) e modificadores de sentença.

Agora, para realizarmos a análise das palavras dentro da frase com a gramática de estrutura frasal, precisamos utilizar o conceito de **sintagma**.



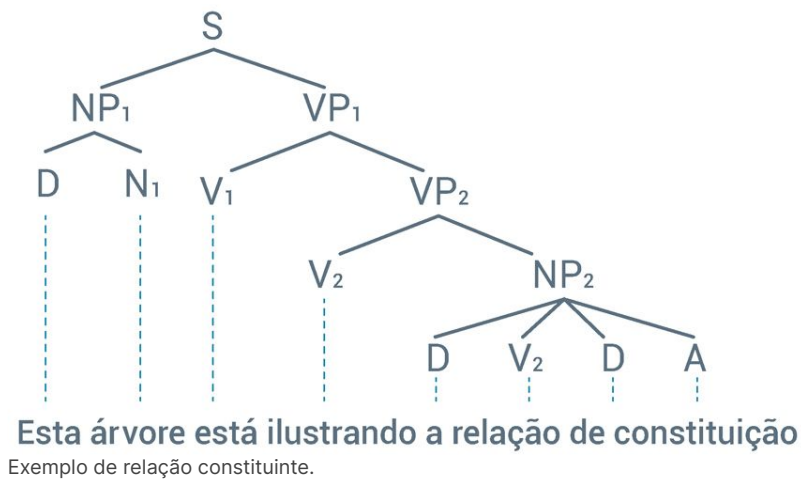
Estudo da língua portuguesa.

Um sintagma é definido como uma estrutura binária composta, obrigatoriamente, por um núcleo e, opcionalmente, por um periférico.

Confira os sintagmas que vamos utilizar!

- NP: representa uma frase nominal.
- VP: representa uma oração que, obrigatoriamente, é constituída por um núcleo verbal.
- N: representa um nome.
- D: é o determinante de um nome. Na língua portuguesa, utilizamos os artigos para determinar uma palavra.

Por exemplo, vamos considerar a seguinte frase: “Esta árvore está ilustrando a relação de constituição”. Vejamos, a seguir, sua estrutura frasal.



A seguir, vejamos alguns exemplos de gramática de estrutura frasal.

Exemplos da gramática de estrutura frasal

Na gramática de estrutura frasal, podemos utilizar quaisquer constituintes para organizar as palavras dentro de uma frase através do uso de suas propriedades. Por exemplo, vamos considerar a seguinte gramática de estrutura frasal:

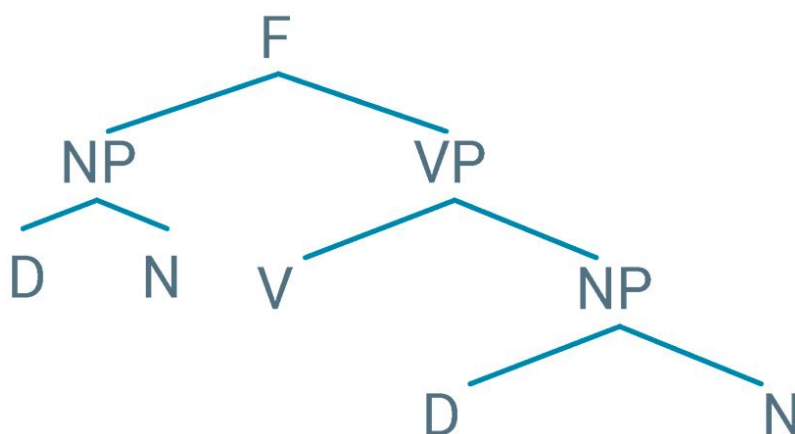
$$F \rightarrow NP VP$$

$$NP \rightarrow D N$$

A representação das variáveis é dada por:

- F : frase ou sentença.
- NP : sintagma nominal.
- VP : sintagma verbal.
- N : nome.
- D : determinante.

Agora, vamos considerar a frase: “O cachorro está deitado no sofá”. Dessa forma, podemos aplicar a gramática que acabamos de definir e obter o seguinte resultado dado na imagem a seguir.



O cachorro está deitado no sofá

Árvore da aplicação da gramática de estrutura frasal.

A gramática de estrutura frasal é bastante flexível. Vejamos um outro exemplo em que podemos aplicá-la para organizar uma frase com os seguintes constituintes – um sujeito, um contexto e um objeto, ou seja:

$$F \rightarrow \langle \text{sujeito} \rangle \langle \text{contexto} \rangle \langle \text{objeto} \rangle$$

Agora, vamos delimitar o escopo dos valores que esses constituintes podem assumir:

⟨sujeito⟩

Os professoresOs estudantesEles

⟨contexto⟩

estão estudandoestão ensinandoestão
descansando

⟨objeto⟩

na bibliotecana sala de auladesde cedo

Nesse exemplo, podemos gerar até 27 frases, como:

- “Os professores estão ensinando na sala de aula”
- “Os estudantes estão estudando na biblioteca”
- “Eles estão descansando desde cedo”

Chegou o momento de aplicarmos nossos conhecimentos para resolver o problema proposto e consolidarmos o que aprendemos. Vamos lá?

Atividade 4

A escrita é uma das formas de expressar os elementos de uma linguagem. Há muitas situações em que textos podem conter informações relevantes, no entanto, não é trivial obtê-las, principalmente se estamos trabalhando com um grande volume de dados. Nesse sentido, selecione a opção correta que apresenta um grande desafio relacionado ao uso prático da gramática de estrutura frasal.

A

Descrever as regras de produção para textos técnicos.

B

Identificar os termos constituintes da linguagem formal.

C

Gerar frases com base no domínio dos elementos constituintes.

D

Conseguir fazer representações realísticas da linguagem.

E

Representar os elementos constituintes de termos coloquiais.



A alternativa E está correta.

A gramática de estrutura frasal é uma ferramenta muito útil para identificar e descrever os elementos que formam uma linguagem. Ela pode ser aplicada em diversos contextos, no entanto, é um grande desafio utilizá-la para representar termos coloquiais, pois esses termos podem estar relacionados a um conteúdo ambíguo, em que a frase pode ter mais de uma interpretação válida.

Aplicações de gramática para PLN

Confira, neste vídeo, aplicações de gramática para processamento de linguagem natural (PLN). Através desses exemplos, podemos ter uma compreensão mais clara sobre a importância prática das gramáticas na resolução de problemas do dia a dia.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aplicações da PLN

Podemos encontrar muitas aplicações práticas de estruturas gramaticais e formalismos para tratar do processamento de linguagem natural (PLN). Entre essas aplicações, temos alguns elementos importantes. Vamos conferir!

Análise e compreensão

As gramáticas permitem que façamos a análise de frases de forma eficiente para determinar se estão ou não corretas do ponto de vista sintático e apresentar o que elas significam.

Geração de linguagem

O uso de gramáticas nos habilita a construir frases que sigam os padrões de linguagem que queremos trabalhar.

Modelagem de linguagem

Através da modelagem de uma linguagem por gramáticas, podemos detectar os padrões sintáticos e semânticos da linguagem natural.

Deteção e correção de erros

Os sistemas de processamento de linguagem natural podem utilizar gramáticas para detectar inconsistências. Além disso, podem apresentar sugestões para correção de erros ou melhoria da qualidade geral do texto.

Padronização e interoperabilidade

Ao usarmos as gramáticas, criamos um ambiente favorável para padronizar a representação da estrutura da linguagem. Então, se aplicarmos um formalismo gramatical comum, podemos realizar trocas com outros sistemas que utilizem o mesmo padrão de gramática.

Como vimos, as estruturas gramaticais e os formalismos são instrumentos muito importantes para o processamento de linguagem natural, pois a combinação de ambos nos oferece os recursos necessários para realizar análises, geração e modelagem da linguagem natural. A seguir, vamos conhecer algumas aplicações bastante interessantes de processamento de linguagem natural.

Mais algumas aplicações de PLN

Já tivemos uma visão geral do uso das estruturas gramaticais e formalismos para o processamento de linguagem natural. Agora, vamos conhecer melhor algumas aplicações.

Reconhecimento de entidades nomeadas

Ao lermos um texto, podemos identificar pessoas, objetos e lugares, por exemplo. Esses elementos são conhecidos como **entidades**.

O nosso objetivo é reconhecer as entidades de forma automatizada e nomeá-los. Esse trabalho é bastante complexo sem a utilização de gramáticas, pois, normalmente, trabalhamos com textos não estruturados.

Sistema de perguntas e respostas

A gramática é bastante útil para construirmos sistemas de perguntas e respostas. Entre as diversas maneiras que podemos utilizar as gramáticas para esse tipo de sistema estão alguns itens primordiais. Vejamos!

1

Análise de perguntas

Podemos analisar a estrutura gramatical das perguntas para identificar o tipo delas e extrairmos informações relevantes, tais como regras e padrões gramaticais que formam as partes constituintes.

2

Extração de respostas

Ao compreendermos a pergunta, podemos fazer consultas com maiores chances de sucesso à nossa base de conhecimento.

3

Geração de respostas

A utilização das gramáticas nos permite formular respostas coerentes com as perguntas através da aplicação de regras precisas e que estejam em conformidade com as regras do idioma.

4

Tratamento de erros

Podemos usar a gramática para identificar possíveis erros gramaticais nas consultas dos usuários e propormos sugestões para melhorar a experiência deles com o sistema.

Resumo de texto

Podemos usar as gramáticas para:

1

Analisar as frases de um documento e identificar as que são mais importantes. Dessa forma, podemos extrair as frases mais relevantes que precisam ser incluídas no resumo.

2

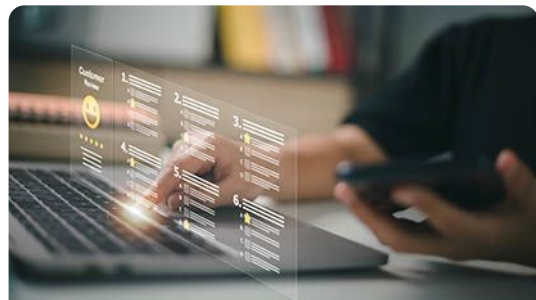
Fazer a compressão de frases preservando o significado central do texto. Assim, obtemos resumos mais concisos sem prejuízo do significado das informações.

Classificação de texto

Atualmente, é muito comum que os usuários de serviços e produtos façam comentários sobre a opinião deles em blogs ou sites.

Para os fornecedores, fabricantes e prestadores de serviço, essas opiniões podem ser bastante relevantes, principalmente, quando há volume significativo. Esses comentários podem expressar sentimentos positivos, ou negativos, ou, ainda, neutros. É o que chamamos de **classificação de sentimentos**.

Além da classificação de sentimentos, também há outras classificações de texto importantes, como detecções de idioma, fraudes, abuso on-line e urgência no apoio ao cliente.



Classificação de sentimentos.

Essas foram apenas algumas das aplicações que podemos realizar com o uso de gramáticas. Cada uma delas tem suas particularidades e exigem maior aprofundamento. Nesse momento, é importante que tenhamos uma boa compreensão sobre a utilidade delas e como estão relacionadas com o uso de estruturas gramaticais e formalismos. Agora, chegou o momento de exercitar o nosso conhecimento. Bom trabalho!

Atividade 5

As estruturas gramaticais e os formalismos são instrumentos fundamentais no processamento de linguagem natural (PLN). Um fato que apoia essa afirmação é a grande quantidade de aplicações que podemos tratar com o uso desses instrumentos. Nesse sentido, selecione a opção que contém uma afirmação correta sobre as aplicações de processamento de linguagem natural com o uso de estruturas gramaticais e de formalismos.

A

As gramáticas sempre devem realizar correções sobre a entrada de dados do usuário em um sistema de PLN.

B

Os formalismos dispensam o uso de gramáticas em aplicações de PLN.

C

Uma forma muito útil de aplicar gramáticas é na transformação dos dados que estão em tabelas de forma automatizada sem a intervenção de um responsável.

D

As gramáticas são a única forma de trabalharmos com PLN.

E

As gramáticas podem ser utilizadas com sucesso para realizar buscas com precisão.



A alternativa E está correta.

As gramáticas são um importante instrumento para melhorar a interação entre os usuários e os sistemas. Um exemplo disso ocorre quando os usuários fazem consultas no sistema. O algoritmo de PLN pode utilizar técnicas que apresentem para os usuários formas mais eficientes de realizar uma consulta. No entanto, é importante que o sistema funcione como um instrumento auxiliar, deixando o protagonismo da escolha para o usuário.

Visão geral da NLTK

Confira, neste vídeo, o que é a biblioteca NLTK e observe como ela é útil para o processamento de linguagem natural. Veja também a preparação do ambiente de desenvolvimento, o que é extremamente necessário para o uso dessa biblioteca.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Aspectos conceituais sobre a NLTK

A NLTK é uma sigla para *Natural Language Toolkit*, que pode ser traduzida, literalmente, para o português como Conjunto de Ferramentas para Linguagem Natural.

O nome da NLTK é bastante sugestivo. Trata-se de uma biblioteca da linguagem de programação Python cujo objetivo é trabalhar com dados de linguagem humana. Portanto, podemos encontrar diversas funcionalidades voltadas para tarefas de processamento de linguagem natural (PLN).

A NLTK é uma biblioteca bastante popular para PLN, pois ela nos permite tratar aspectos de processamento e análise de texto. Além disso, ela possui características interessantes que abrangem os seguintes aspectos:

Código aberto

O objetivo é que pesquisadores e estudantes tenham acesso gratuito a ferramentas de PLN para facilitar o aprendizado e os testes. Inclusive, existem diversos exemplos e tutoriais disponíveis sobre como usar a NLTK.

Processamento linguístico

Trata de assuntos específicos de PLN, como morfologia, sintaxe e semântica.

Extensibilidade

A NLTK foi projetada para viabilizar a adição de novos algoritmos, modelos e recursos.

Ao usarmos a NLTK, estamos trabalhando com uma biblioteca que é referência em PLN e adiciona-se a isso o fato de estarmos colaborando com a comunidade de pesquisa através da troca de experiências e algoritmos.

Preparação de ambiente da NLTK

Os primeiros pesquisadores que desenvolveram a NLTK foram Steven Bird e Edward Loper no começo dos anos 2000 na Universidade da Pensilvânia. Em seguida, eles fizeram o lançamento em 2001. Depois disso, a NLTK, como podíamos esperar, passou por várias versões e foi diversas vezes atualizada.

Atualmente, as melhores versões do Python para usar a NLTK são 2.7, ou 3.4, ou superior. Nesse trabalho, vamos usar o Google Colab para implementar nossos exemplos. Então, nosso primeiro passo é testar a versão do Python no ambiente do Google Colab. Para isso, precisamos digitar a seguinte linha de comando:

```
python
!python --version
```

No nosso caso, a versão do Python que estamos usando é 3.10.12. Agora, precisamos instalar a NLTK. Então, nosso próximo passo é digitar a seguinte linha de comando:

```
python
!pip install nltk
```

O Colab vai nos exibir os resultados da instalação, conforme a imagem a seguir.

```
!pip install nltk
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (3.8.1)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk) (8.1.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk) (1.2.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk) (2022.10.31)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from nltk) (4.65.0)
```

Resultado da instalação da NLTK.

Como podemos perceber na imagem, a instalação funcionou corretamente. Agora, precisamos verificar a versão da NLTK. Para isso, precisamos importá-la através da execução da linha de comando a seguir.

```
python
import nltk
```

Após, devemos executar a seguinte linha de comando:

```
python
print('A versão da NLTK é { }.'.format(nltk.__version__))
```

No nosso caso, o resultado da execução foi o seguinte: A versão do NLTK é 3.8.1.

É importante que você tente realizar esses passos para começar a se familiarizar com a NLTK, pois vamos precisar que o ambiente esteja configurado corretamente para desenvolvermos os nossos exemplos. Agora, chegou o momento de fazermos nosso exercício para consolidar nossos conhecimentos. Vamos lá!

Atividade 1

O processamento de linguagem natural (PLN) é um assunto que apresenta diversos desafios. Uma das implicações dessa afirmação é que implementar todos os algoritmos que precisamos poderá inviabilizar o desenvolvimento de um projeto de PLN. Nesse sentido, selecione a opção correta a respeito da biblioteca NLTK.

A

A NLTK é uma biblioteca desenvolvida para trabalhar com qualquer linguagem de programação.

B

A NLTK oferece funcionalidades que não têm como foco a eficiência, mas a obtenção de resultados corretos.

C

A NLTK provê apenas os aspectos conceituais dos algoritmos de PLN, ficando sob a responsabilidade do programador o desenvolvimento deles.

D

O uso da NLTK para aplicações de PLN é um caso particular, uma vez que o escopo dela é muito maior.

E

O fato de a NLTK ser bem documentada influenciou bastante sua popularidade.



A alternativa E está correta.

A NLTK é uma biblioteca desenvolvida para linguagem de programação Python cujo foco é o processamento de linguagem natural. Ela disponibiliza diversas funções para tratamento de textos que facilitam o desenvolvimento de soluções eficientes. Além disso, a NLTK possui uma rica documentação, que foi um dos pontos cruciais para que ela ganhasse muita popularidade.

Tokenização e marcação de parte da fala

Confira, neste vídeo, duas importantes técnicas de processamento de linguagem natural: tokenização e marcação de parte da fala. Ambas são úteis para extração de valor dos dados de textos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Tokenização

Quando “quebramos” um texto em unidades menores que tenham significado, estamos realizando o processo de **tokenização**. Por isso, essas unidades menores são chamadas de **tokens**. Normalmente, quando trabalhamos com PLN, realizamos a tokenização de um texto para obtermos palavras ou frases individuais.

No contexto dos algoritmos de PLN, realizamos a tokenização no pré-processamento do texto, pois ganhamos mais flexibilidade de manipular os tokens para extrair informações com maior valor agregado. Veja agora os tipos mais comuns de tokenização!

1

Tokenização de palavras

Fazemos a segmentação de palavras, ou seja, aplicamos a divisão de um texto em palavras individuais. Por exemplo, vamos considerar a frase: “O estudo das gramáticas é essencial para PLN”. Agora, depois de aplicar a tokenização de palavras, obtemos:

["O", "estudo", "das", "gramáticas", "é", "essencial", "para", "PLN"]

2Tokenização de frases

Temos como objetivo identificar os limites das frases, considerando os sinais de pontuação. Por exemplo, vamos considerar o seguinte trecho: “A PLN faz uso de diversas técnicas. Uma dessas técnicas é a tokenização”. Agora, vamos aplicar a tokenização de frases e, assim, obtemos o resultado:

[“A PLN faz uso de diversas técnicas.”, “Uma dessas técnicas é a tokenização”]

Podemos aplicar a tokenização em muitas tarefas de PLN, como classificação de texto, recuperação de informações, análise de sentimento, tradução automática, reconhecimento de entidade nomeada, por exemplo. O fato de trabalharmos com tokens facilita realizarmos análises mais sofisticadas. A NLTK oferece alguns métodos de tokenização, como:

`word_tokenize()`

Tokenização de palavras.

`sent_tokenize()`

Tokenização de frases.

Na sequência, vamos estudar mais uma técnica de PLN e entender como a NLTK pode nos ajudar a utilizá-la.

Marcação de parte da fala

É o processo de rotularmos cada palavra de uma frase. Na prática, isso significa dizer que estamos fazendo indicação da classe gramatical e função sintática de cada palavra dentro da frase. Ela também é conhecida como **marcação gramatical** ou **desambiguação de categoria de palavra**. A expressão original em inglês é *part-of-speech* (POS) *tagging* e a sigla POS é bastante utilizada para fazermos referência a essa técnica.

As classes gramaticais das palavras são: substantivos, artigos, adjetivos, pronomes, verbos, numerais, advérbios, preposições, conjunções e interjeição. Normalmente, usamos a palavra determinante para nos referenciar a um artigo. São essas classes que usamos para fazer a marcação POS. Elas fornecem informações linguísticas que são úteis tanto na análise sintática como na semântica. Vamos considerar a seguinte frase:

“O cachorro está passeando com o filhote.”

Agora, vamos aplicar as tags da marcação de fala. Confira o resultado obtido na tabela a seguir.

Palavra	Tag
O	Determinante
cachorro	Substantivo
está	Verbo
passeando	Verbo
com	Preposição
o	Determinante

Palavra	Tag
filhote	Substantivo
.	Pontuação

Tabela: Aplicação das tags da POS.
Sérgio Assunção Monteiro.

A marcação de parte da fala serve como base para várias tarefas de análise de linguagem natural, como compreensão do texto, extração de informações, análise de sentimento, tradução automática e análise sintática, a qual nos ajuda a entender a estrutura hierárquica das frases. A NLTK possui funções para marcação de POS, como a função ``pos_tag()'`, que atribui tags POS a palavras.

Agora que já sabemos o que são as técnicas de tokenização e marcação de fala, vamos fazer um exercício para fixar os nossos conhecimentos. Bons estudos!

Atividade 2

Cada vez mais, vemos a importância de extrair valor dos dados. Isso ocorre em vários tipos de situações. Em alguns casos, esse desafio é bastante complexo, como o que acontece com o tratamento de textos em linguagem natural. Nesse sentido, selecione a opção correta que explica o papel da tokenização na extração de valor de dados em textos em linguagem natural.

A

Podemos identificar elementos e, assim, usá-los para fazer tratamentos específicos.

B

O principal benefício que ela nos dá é a identificação das regras sintáticas.

C

Ela faz correções automáticas nos textos.

D

Um dos efeitos indesejados dela é tratar palavras e frases da mesma forma.

E

Os algoritmos de tokenização sempre devem ser implementados pelo desenvolvedor.



A alternativa A está correta.

A tokenização pode ser usada para obter a separação de palavras ou de frases. Em cada um desses casos, devemos usar uma técnica distinta. Uma das grandes vantagens é podermos utilizar os resultados para identificar entidades e, assim, darmos um tratamento específico para elas.

Stemming, lematização e reconhecimento de entidades nomeadas

Confira, neste vídeo, três técnicas que são muito usadas nas aplicações de processamento de linguagem natural: stemming, lematização e reconhecimento de entidades nomeadas.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Stemming e Lematização

Nós utilizamos as técnicas de stemming (derivação) e lematização no processamento de linguagem natural (PLN) com o objetivo de reduzir as palavras para as suas formas de base ou raiz. A ideia é diminuir o impacto das variações de palavras para melhorar a recuperação de informações e simplificar a análise de texto. As duas técnicas têm objetivos semelhantes, mas possuem abordagens diferentes.

A biblioteca NLTK possui algoritmos para stemming e lematização. Para os algoritmos de stemming, ela oferece métodos como Stemmers Porter e Snowball. No caso da lematização, a NLTK usa o WordNet. A seguir, vamos conhecer um pouco mais sobre as técnicas de stemming e lematização.

Stemming

É uma técnica para reduzir as palavras à sua raiz ou radical, removendo prefixos, sufixos e outros afixos. Vejamos um exemplo da aplicação dessa técnica:

Palavra original

andando



Porter stemmer

and

O principal cenário em que essa técnica é útil ocorre nos mecanismos de pesquisa.

Lematização

É aplicada para encontrar a forma base de uma palavra usando regras linguísticas e análise morfológica. Essa técnica considera o contexto da palavra e a classe gramatical para produzir uma forma básica válida e significativa. Vejamos um exemplo da aplicação dessa técnica:

Palavra original

andando



Lema

anda

Utilizamos a lematização quando precisamos de precisão na compreensão das palavras, como nas aplicações de compreensão de linguagem, análise de texto e tradução automática.

Reconhecimento de entidades nomeadas

É utilizado para identificar e categorizar entidades nomeadas em texto. As entidades nomeadas podem ser objetos, pessoas, organizações, locais, datas, quantidades e valores monetários, por exemplo.

Vamos analisar uma situação em que aplicamos a técnica de entidades nomeadas. Considere a frase a seguir:

“A rede de lojas XYZ anunciou um plano de expansão de novas filiais no Rio de Janeiro no segundo semestre.”

Anotações NER (reconhecimento de entidade nomeada):

- Entidade nomeada: “Rede de lojas XYZ” – Tipo de entidade: Organização
- Entidade nomeada: “Rio de Janeiro” – Tipo de entidade: Localização
- Entidade nomeada: “segundo semestre” – Tipo de entidade: Data

A técnica de reconhecimento de entidade nomeada permite que possamos extrair informações estruturadas de texto não estruturado. Assim, podemos realizar uma análise mais profunda dos dados textuais. A NLTK nos fornece ferramentas para reconhecimento de entidades nomeadas.

Agora, chegou a hora de consolidarmos nossos conhecimentos com a realização de mais um exercício.

Atividade 3

A compreensão das palavras é um aspecto essencial da nossa comunicação. Um dos desafios do processamento de linguagem natural é sistematizar esse processo de compreensão. Nesse sentido, selecione a opção correta a respeito das técnicas de stemming, lematização e reconhecimento de entidades nomeadas.

A

As três técnicas são usadas nos mesmos cenários.

B

As técnicas de stemming e lematização produzem resultados iguais.

C

A dificuldade para obter os resultados das três técnicas é equivalente.

D

O reconhecimento de entidades nomeadas é mais complexo do que obter lematização de palavras.

E

A NLTK fornece funcionalidades apenas para o reconhecimento de entidades nomeadas.



A alternativa D está correta.

As técnicas de stemming, lematização e reconhecimento de entidades nomeadas são importantes para as aplicações de processamento de linguagem natural. Cada uma delas tem um contexto mais adequado para ser utilizada, no entanto, a que produz resultados mais elaborados das três é a técnica de reconhecimento de entidades nomeadas.

Análise de sentimentos

Confira, neste vídeo, os aspectos conceituais e as etapas e aplicações de uma das aplicações mais populares do processamento de linguagem natural: a análise de sentimentos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceitos sobre a análise de sentimentos

A análise de sentimentos é uma das aplicações mais populares do processamento de linguagem natural. Ela tem como objetivo determinar a informação subjetiva que está sendo expressa em um texto. Não à toa, ela também é conhecida como mineração de opinião, pois nos permite analisar as opiniões e emoções dentro de um texto. Uma das formas mais simples que podemos realizar essa análise é através da utilização de categorias de sentimento, como:

Neutro

Positivo

Negativo

Vamos considerar um exemplo:

Frase

O filme foi excelente!



Resultado da análise de sentimentos

Positivo.

Há situações em que podemos refinar ainda mais as categorias de sentimentos. No entanto, o ponto mais importante para observarmos é que saímos de um cenário em que trabalhávamos com uma entrada de dados como texto para um resultado categórico que nos permite fazer análises objetivas.

Etapas da análise de sentimentos

As técnicas de análise de sentimento passam pelas seguintes etapas, vejamos!

Pré-processamento de texto

Etapa em que realizamos a limpeza do texto para remover pontuação, palavras irrelevantes ou caracteres irrelevantes. Além disso, também podemos aplicar as técnicas de stemming e lematização para facilitar o processamento das palavras nas próximas etapas.

Extração de características

Etapa em que temos como objetivo obter elementos que permitam fazer a identificação de características relevantes para identificar as categorias de sentimentos nas próximas etapas.

Aplicação de algoritmos e modelos para classificação dos sentimentos

Etapa em que temos várias técnicas de inteligência artificial à nossa disposição, como algoritmos de aprendizado de máquina e modelos de aprendizado profundo que precisam ser treinados e podem competir entre si para produzir resultados de boa qualidade.

Saída da análise de sentimento

Etapa em que apresentamos o resultado obtido em categorias.

Na sequência, vamos estudar algumas das aplicações que podemos realizar com a análise de sentimentos.

Aplicações da análise de sentimentos

A quantidade de aplicações em que podemos usar a análise de sentimentos é gigantesca. Um dos principais fatores que influenciaram deve-se à grande popularidade das redes sociais. Entre as aplicações da análise de sentimentos, temos:

Análise de mídia social

A intenção é analisar os sentimentos expressos em postagens e comentários nas redes sociais.

Monitoramento de produtos, serviços e marcas

O objetivo é avaliar o sentimento relacionado a determinados produtos, serviços e marcas.

Pesquisa de mercado

A ideia é estudar as opiniões de clientes para obter informações sobre preferências e sentimentos.

Gerenciamento de reputação

A ideia é avaliar a opinião pública para gerenciar a reputação de indivíduos e organizações que tenham grande exposição.

Mais um ponto importante sobre a análise de sentimentos é que ela nos ajuda a entender e resumir grandes volumes de texto. A biblioteca NLTK possui módulos para análise de sentimentos.

Atividade 4

Nos últimos anos, tem ficado cada vez mais comum que haja uma maior interação entre empresas prestadoras de serviço e consumidores. Um dos meios em que essa interação ocorre é através de chats. Nesse sentido, selecione a opção correta sobre a aplicação da análise de sentimentos no contexto de chats.

A

A análise de sentimentos elimina a necessidade de se ter um atendente humano para interagir com os clientes.

B

A análise de sentimentos utiliza técnicas robustas que sempre produzem resultados totalmente confiáveis.

C

Desenvolver um algoritmo de análise de sentimentos é uma tarefa muito simples.

D

A única maneira de desenvolver um método de análise de sentimentos é através da busca de palavras-chaves dentro de uma frase.

E

As informações sobre o estado emocional do consumidor podem direcionar a interação entre o atendente e o consumidor.



A alternativa E está correta.

A análise de sentimentos é uma das aplicações mais populares do processamento de linguagem natural. Esse tipo de aplicação utiliza de várias técnicas de inteligência artificial para realizar a categorização de sentimentos em textos não estruturados. Uma situação em que ela pode ser aplicada é no tratamento do texto de chats e, assim, fornecer informações para o prestador de serviço sobre o modo que os consumidores se expressam que serão úteis para aperfeiçoar o atendimento.

WordNet e corpus

Confira, neste vídeo, os aspectos conceituais e um exemplo da gramática livre de contexto (GLC). A GLC é o ponto inicial para entendermos, na prática, como podemos formalizar as regras sintáticas de uma linguagem e aplicar algoritmos para processamento de linguagem natural.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

WordNet

É um conjunto de dados que traz informações em dois aspectos relevantes. Confira!

Léxico

O foco é a palavra em si mesma.

Semântico

O significado da palavra é interpretado através dos seus relacionamentos.

A WordNet apresenta a organização das palavras em conjuntos de sinônimos conhecidos como synsets. Dessa forma, temos uma estrutura hierárquica rica que expressa as várias relações semânticas entre as palavras. Vejamos!

Hiperonímia

Trata das palavras no sentido mais geral. Por exemplo, a palavra “animal” é um hiperônimo de “leão”.

Hiponímia

Trata das palavras no sentido mais específico. Por exemplo, a palavra “leão” é um hipônimo de “animal”.

Meronímia

Trata da relação parte-todo. Por exemplo, a palavra “dentes” tem uma relação com a palavra “boca”.

Sinonímia

Apresenta uma relação de significado semelhante. Por exemplo, as palavras “engraçado” e “cômico” possuem uma relação de sinonímia.

Antonímia

Apresenta uma relação de significado oposto. Por exemplo, as palavras “fortaleza” e “fraqueza” possuem uma relação de antonímia.

Essas relações são úteis para que os algoritmos de processamento de linguagem natural possam explorar os significados das palavras. Por isso, a WordNet é usada em várias situações práticas que envolvem:

- Redução de ambiguidade de palavras
- Recuperação de informações
- Classificação de texto
- Análise de sentimento

A biblioteca NLTK fornece funcionalidades para que tenhamos acesso aos synsets do WordNet.

Corpus

Usamos esse termo para nos referirmos a uma coleção estruturada de textos para realizarmos:

- Análise linguística
- Modelagem de linguagem
- Análise estatística

Essa coleção deve ter relevância estatística para que obtenhamos a representatividade de uma língua ou de determinado domínio. Assim, podemos estudar padrões, frequências e fenômenos linguísticos.

As fontes de dados para construirmos corpus podem ser diversas, como livros, artigos e sites, por exemplo. Inclusive podemos utilizar informações linguísticas, como tags de parte do discurso, entidades nomeadas, estruturas sintáticas e rótulos de sentimento para prover mais elementos relevantes aos algoritmos de processamento de linguagem natural.

O corpus é útil para fazermos o treinamento e a avaliação dos modelos de linguagem. A biblioteca NLTK nos fornece funcionalidades para trabalharmos com corpus de textos.



Livros como fontes de dados corpus.

Atividade 5

Um dos desafios do processamento de linguagem natural é tratar a ambiguidade. Uma forma de realizar esse tratamento é analisar o contexto das palavras. Nesse sentido, selecione a opção que contém uma afirmação correta sobre o uso de WordNet para reduzir a ambiguidade.

A

A WordNet identifica as palavras de uma forma única através da análise contextual em que elas aparecem.

B

A WordNet fornece relações entre as palavras que auxiliam na redução da ambiguidade entre elas.

C

A WordNet é um algoritmo de processamento de linguagem natural que faz uso de técnicas de inteligência artificial para classificar as palavras.

D

A WordNet utiliza as técnicas de stemming e lematização para eliminar o sentido ambíguo da interpretação das palavras.

E

A WordNet é um método que consome muitos recursos computacionais e, na prática, funciona apenas como um modelo para desenvolvermos algoritmos mais simples de processamento de linguagem natural.



A alternativa B está correta.

A WordNet é um recurso que oferece as relações sintáticas entre as palavras. Dessa forma, ela pode ser utilizada por técnicas de processamento de linguagem natural para melhorar a precisão da classificação de textos.

Tokenização de palavras com NLTK

Nesta prática, você terá a chance de demonstrar seus conhecimentos para aplicar a biblioteca NLTK no Python para fazer a tokenização de palavras dentro de um texto. Assista ao vídeo!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você aplique a técnica de tokenização em um texto através da biblioteca NLTK usando o Python. Para alcançar esse objetivo, siga as seguintes etapas.

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:

```
python
!pip install nltk
```

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
import nltk
nltk.download('punkt')
```

O “punkt” é um tokenizador de palavras da biblioteca NLTK.

Lembre-se: O aninhamento do código faz parte da sintaxe do Python. Caso o código que você escrever não esteja aninhado corretamente, seu programa não vai executar.

Passo 5

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:


```
python

from nltk.tokenize import word_tokenize
texto = "Eu estudo processamento de linguagem natural."
tokens = word_tokenize(texto)
print(tokens)
```

O resultado da execução é dado por:

```
python

['Eu', 'estudo', 'processamento', 'de', 'linguagem', 'natural', '.']
```

A lista resultante contém todas as palavras do nosso texto. Essa é uma aplicação muito comum nas primeiras etapas do processamento de linguagem de linguagem natural. Também poderíamos obter um resultado semelhante utilizando expressões regulares, mas, nesse momento, queremos nos familiarizar melhor com a biblioteca NLTK.

Agora, é a sua vez de fazer um exercício que vai ajudar a consolidar os conhecimentos.

Atividade 1

Uma das aplicações que envolvem PLN tem a ver com aplicar filtros no texto. Por exemplo, podemos selecionar palavras que tenham um comprimento mínimo. Nesse sentido, implemente um programa em Python utilizando a biblioteca NLTK para imprimir apenas as palavras com comprimento maior ou igual a 5. Como dica, comece seu código da seguinte forma:

```
python

from nltk.tokenize import word_tokenize
texto = "Eu estudo processamento de linguagem natural."
```

A resposta deve ser:

```
estudo
processamento
linguagem
natural
```

Chave de resposta

A solução do problema é dada por:

```
python

from nltk.tokenize import word_tokenize
texto = "Eu estudo processamento de linguagem natural."
tokens = word_tokenize(texto)
palavras_filtradas = [palavra for palavra in tokens if len(palavra) > 4]
for p in palavras_filtradas:
    print(p)

from nltk.tokenize import word_tokenize
texto = "Eu estudo processamento de linguagem natural."
tokens = word_tokenize(texto)
palavras_filtradas = [palavra for palavra in tokens if len(palavra) > 4]
for p in palavras_filtradas:
    print(p)
```

Cuja resposta é:

```
estudo
processamento
linguagem
natural
```

Definição de gramática com NLTK

Nesta prática, você vai desenvolver uma gramática para reconhecer expressões aritméticas com o uso da biblioteca NLTK. Assista ao vídeo!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você utilize as funcionalidades da biblioteca NLTK para criar uma gramática a fim de validar expressões aritméticas que envolvam exclusivamente as operações de adição, subtração, multiplicação e divisão. Lembre-se de que é essencial garantir que a biblioteca NLTK esteja instalada, como fizemos na prática anterior. Para alcançar o nosso objetivo, vamos seguir as seguintes etapas no Google Colab.

Passo 1

Escreva e execute, exatamente, o seguinte código:

```
python

import nltk
```

Passo 2

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
```

```
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
```

Para definir a gramática, tivemos que utilizar a funcionalidade *“nltk.CFG.fromstring”*. Outro ponto importante que devemos observar são as letras à esquerda das setas que indicam as variáveis de produção. Já as combinações das letras à direita das setas representam as regras de produção.

Passo 3

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
```

```
print("Produções: ", gramatica.productions())
```

Você acabou de usar a função “productions” da biblioteca NLTK, cujo objetivo é nos dar acesso às regras de produção da gramática. O resultado da execução é dado por:

```
python
```

```
Produções: [Expressao -> Numero, Expressao -> A, Expressao -> S, Expressao -> M,
Expressao -> D, A ->
Expressao '+' Expressao, S -> Expressao '-' Expressao, M -> Expressao '*' Expressao, D ->
Expressao '/'
Expressao, Numero -> Digito, Numero -> Digito Numero, Digito -> '0', Digito -> '1',
Digito -> '2',
Digito -> '3', Digito -> '4', Digito -> '5', Digito -> '6', Digito -> '7', Digito -> '8',
Digito -> '9']
```

Ou seja, acabamos de imprimir todas as regras de produção da nossa gramática. Agora, faça um exercício para ajudar a consolidar seus conhecimentos.

Atividade 2

As gramáticas são instrumentos muito úteis para representarmos regras sintáticas. Normalmente, elas fazem uso de estruturas recursivas que permitem a generalização de padrões. Nesse sentido, implemente um programa em Python utilizando a biblioteca NLTK para imprimir apenas a segunda regra de produção da gramática estudada que reconhece expressões aritméticas.

A resposta deve ser “Segunda regra de produção: A → Expressao '+' Expressao”.

Chave de resposta

Uma das formas de resolver esse problema é através do seguinte código:

```
python

import nltk
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
print("Segunda regra de produção: ", gramatica.productions()[5])
```

Cuja resposta é:

Segunda regra de produção: $A \rightarrow \text{Expressao '+' Expressao}$.

Validação sintática com a NLTK

Nesta prática, você vai testar se determinada entrada de dados satisfaz as regras sintáticas descritas em uma gramática. Assista ao vídeo!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você aplique as funcionalidades da biblioteca NLTK para fazer a validação sintática de uma expressão aritmética. Para alcançar o nosso objetivo, vamos seguir as seguintes etapas no Google Colab:

Passo 1

Escreva e execute, exatamente, o seguinte código:

```
python

import nltk
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
```

Ou seja, acabamos de definir a gramática como no exemplo anterior.

Passo 2

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python

print("*** Processamento ***")
tokens = "1 + 2".split()
cp = nltk.ChartParser(gramatica)
for tree in cp.parse(tokens):
    print(tree)
```

Nesse trecho de código, entramos com a expressão aritmética "1 + 2". Em seguida, criamos um objeto "cp" para realizar a validação sintática da nossa entrada de acordo com as regras de produção da nossa gramática. O resultado da execução é dado por:

```
python

*** Processamento ***
(Expressao
 (A
  (Expressao (Numero (Digito 1)))
  +
  (Expressao (Numero (Digito 2)))))
```

Esse resultado indica os caminhos que precisamos percorrer para validar a entrada de dados. Agora, é a sua vez de fazer um exercício para consolidar seus conhecimentos.

Atividade 3

As gramáticas são fundamentais para representarmos regras sintáticas. Depois de implementarmos a implementação delas, precisamos usá-las para fazer a validação das entradas de acordo com as regras que estabelecemos. Nesse sentido, implemente um programa em Python utilizando a biblioteca NLTK para validar a seguinte expressão aritmética: 1 + 2 / 3.

```
*** Processamento ***
(Expressao
 (A
  (Expressao (Numero (Digito 1)))
  +
  (Expressao
   (D
    (Expressao (Numero (Digito 2)))
    /
    (Expressao (Numero (Digito 3)))))
  (Expressao
   (D
    (Expressao
     (A
      (Expressao (Numero (Digito 1)))
      +
      (Expressao (Numero (Digito 2)))))
     /
     (Expressao (Numero (Digito 3)))))
```

Resposta do processamento do código.

Chave de resposta

Devido à saída, podemos ser induzidos, erroneamente, que o código para resolver esse problema é bem mais complexo do que vimos. Na verdade, o código é exatamente o mesmo. O único ponto que precisamos mudar é a entrada dos dados. A seguir, mostramos o código completo.

```
python

import nltk
import nltk
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
print("*** Processamento ***")
tokens = "1 + 2 / 3".split()
cp = nltk.ChartParser(gramatica)
for tree in cp.parse(tokens):
    print(tree)
```

Cuja resposta é:

```
*** Processamento ***
(Expressao
 (A
  (Expressao (Numero (Digito 1)))
  +
  (Expressao
   (D
    (Expressao (Numero (Digito 2)))
    /
    (Expressao (Numero (Digito 3)))))
  (Expressao
   (D
    (Expressao
     (A
      (Expressao (Numero (Digito 1)))
      +
      (Expressao (Numero (Digito 2))))
     /
     (Expressao (Numero (Digito 3)))))
    (Expressao (Numero (Digito 3)))))
  (Expressao (Numero (Digito 3)))))
```

Resposta do processamento do código.

Acrescentar regras sintáticas em uma gramática

Nesta prática, você vai se deparar com uma situação bastante comum no uso de gramáticas: a criação de novas regras sintáticas. Assista ao vídeo!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você acrescente uma nova regra sintática em uma gramática. No caso, vamos continuar usando a gramática para representar expressões aritméticas. A regra que vamos adicionar é o reconhecimento de expressões aritméticas entre parênteses. Para alcançar o nosso objetivo, vamos seguir as seguintes etapas no Google Colab.

Passo 1

Escreva e execute, exatamente, o seguinte código:

```
python

import nltk
import nltk
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D | "(" Expressao ")"
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
```

No caso, fizemos o acréscimo da regra que valida a expressão entre parênteses explorando a recursividade das variáveis de produção da gramática.

Passo 2

Agora, crie outra célula de código. Escreva e execute exatamente o seguinte código:

```
python

print("*** Processamento ***")
tokens = "1 + ( 2 / 3 )".split()
cp = nltk.ChartParser(gramatica)
for tree in cp.parse(tokens):
    print(tree)
```

No caso, entramos com a expressão “1 + (2 / 3)”. Como já definimos a regra sintática “expressão entre parênteses”, vamos obter o seguinte resultado:

```

*** Processamento ***
(Expressao
  (A
    (Expressao (Numero (Digito 1)))
    +
    (Expressao
      (
        (Expressao
          (D
            (Expressao (Numero (Digito 2)))
            /
            (Expressao (Numero (Digito 3))))
        )))
  )))

```

Resposta do processamento do código.

Muito bom, não é? Ou seja, acabamos de acrescentar uma regra sintática e já fizemos a validação de uma entrada. É essencial que você tente executar esse exemplo e faça algumas modificações para ter mais segurança. Agora, faça um exercício para consolidar seus conhecimentos.

Atividade 4

Não há dúvidas sobre a importância das gramáticas para aplicações de processamento de linguagem natural. Semelhante ao que ocorre em um software de propósito geral, é natural que as gramáticas também evoluam. Nesse sentido, implemente um programa em Python utilizando a biblioteca NLTK para acrescentar e imprimir a regra sintática “expressão entre colchetes”. Teste o seu código para a seguinte entrada: “(1 + [2 -3])”

A resposta deve ser:

```

*** Processamento ***
(Expressao
  (
    (Expressao
      (A
        (Expressao (Numero (Digito 1)))
        +
        (Expressao
          [
            (Expressao
              (S
                (Expressao (Numero (Digito 2)))
                -
                (Expressao (Numero (Digito 3))))
              ])))
      )))
  ))

```

Resposta do processamento do código.

Chave de resposta

Nesse caso, basta adicionarmos a regra “expressão entre colchetes”, conforme o código a seguir.


```
python

import nltk
gramatica = nltk.CFG.fromstring("""
Expressao -> Numero | A | S | M | D | "(" Expressao ")" | "[" Expressao "]"
A -> Expressao "+" Expressao
S -> Expressao "-" Expressao
M -> Expressao "*" Expressao
D -> Expressao "/" Expressao
Numero -> Digito | Digito Numero
Digito -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
""")
print("*** Processamento ***")
tokens = "( 1 + [ 2 - 3 ] )".split()
cp = nltk.ChartParser(gramatica)
for tree in cp.parse(tokens):
    print(tree)
```

Cuja resposta é:

```
*** Processamento ***
(Expression
 (
  (Expression
   (A
    (Expression (Numero (Digito 1)))
    +
    (Expression
     [
      (Expression
       (S
        (Expression (Numero (Digito 2)))
        -
        (Expression (Numero (Digito 3))))
       ])))
  ))
))
```

Resposta do processamento do código.

Reconhecimento de padrões sintáticos com a NLTK

Nesta prática, você vai utilizar as funcionalidades da NLTK para pesquisar por padrões sintáticos dentro de um texto. Assista ao vídeo!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você encontre um padrão sintático em um texto através das funcionalidades das bibliotecas NLTK e Regex usando o Python. No nosso caso específico, vamos receber como entrada um código na linguagem C e vamos procurar pelo padrão sintático do comando condicional "if". Para alcançar o nosso objetivo, vamos seguir as seguintes etapas no Google Colab.

Passo 1

Escreva e execute, exatamente, o seguinte código:

```
python

import re
from nltk.tokenize import word_tokenize

# Entrada de texto
c_codigo = ''
#include

int main() {
    int num = 5;
    if (num % 2 ==0) {
        printf("Número Par");
    }
    return 0;
}
```

Nesse primeiro trecho, fizemos a entrada do código em C. No caso, o código é simplesmente um texto.

Passo 2

Agora, precisamos criar outra célula de código. Vamos escrever e executar, exatamente, o seguinte código:

```
python

# Tokenizar o código em palavras
tokens = word_tokenize(c_codigo)

# busca da regra sintática if
padrao_sintatico = r"if\s*\([^)]+\)\s*\{[^}]*\}"
sintaxe_if = re.findall(padrao_sintatico, ' '.join(tokens))

for s in sintaxe_if:
    print(s)
```

Logo no início desse trecho de código, fizemos a tokenização da entrada em palavras com o uso da função “word_tokenize” da biblioteca NLTK. Em seguida, fizemos a descrição da regra sintática do comando “if” com o auxílio de uma expressão regular. Quando executamos o código, obtemos o seguinte resultado:

```
if ( num % 2 ==0 ) { printf ( `` Número Par ' ' ) ; }
```

Resposta do processamento do código.

Ou seja, acabamos de extrair todo o trecho de código relacionado ao comando condicional “if”. Agora, é a sua vez de fazer um exercício que vai ajudar a consolidar seus conhecimentos.

Atividade 5

As bibliotecas NLTK e Regex são muito importantes no contexto de processamento de linguagem natural. As funcionalidades dessas bibliotecas permitem que possamos nos concentrar na resolução de problemas ao invés de nos preocuparmos com todos os detalhes da implementação das funções. Nesse sentido, implemente um programa em Python utilizando as bibliotecas NLTK e Regex para receber um código na linguagem C e que extraia o trecho associado ao comando iterativo “while”.

Use o seguinte código em C para testar sua solução:

```
python
#include

int main() {
    int num = 1;
    while (num <= 10) {
        printf("%d ", num);
        num++;
    }
    return 0;
}
```

A resposta deve ser:

```
while ( num <= 10 ) { printf ( `` % d `` , num ) ; num++ ; }.
```

Chave de resposta

A solução desse problema é bastante semelhante à que estudamos. A principal diferença ocorre na expressão regular que usamos para descrever a sintaxe do comando “while”, conforme podemos ver no código a seguir.

```
python
import re
from nltk.tokenize import word_tokenize

c_codigo = ''
#include

int main() {
    int num = 1;
    while (num <= 10) {
        printf("%d ", num);
        num++;
    }
    return 0;
}
'''

# Tokenizar o código em palavras
tokens = word_tokenize(c_codigo)

# Buscar pelo padrão sintático do comando "while"
padrao = r"while\s*\[^\]]+\s*\{[^\}]*\}"
comando_while = re.findall(padrao, ' '.join(tokens))

for s in comando_while:
    print(s)
```

Cuja resposta é:

```
while ( num <= 10 ) { printf ( `` % d `` , num ) ; num++ ; }
```

Considerações finais

- Estruturas gramaticais e formalismos.
- Aplicações de processamento de linguagem natural (PLN).
- Algumas das principais funcionalidades da biblioteca NLTK.
- Implementações de PLN com o uso da NLTK.

Explore +

Confira as indicações que separamos especialmente para você!

- Acesse o site oficial da AWS da Amazon e pesquise por: **O que é análise de sentimentos?** Nele você vai encontrar mais exemplos e outras abordagens sobre aplicações desse assunto.
- Acesse o site oficial da biblioteca NLTK. Lá, você vai encontrar a documentação das funcionalidades da biblioteca e diversos exemplos que vão auxiliar você a se familiarizar com a NLTK.

Referências

BENGFORT, B.; BILBRO, R.; OJEDA, T. **Applied Text Analysis with Python**: Enabling Language-Aware Data Products with Machine Learning. 1. ed. O'Reilly Media, 2018.

BIRD, S.; KLEIN, E.; LOPER, E. **Natural Language Processing with Python**. 1. ed. O'Reilly Media, 2009.

HARDENIYA, N.; PERKINS, J.; CHOPRA, D.; JOSHI, N.; MATHUR, I. **Natural Language Processing**: Python and NLTK. 1. ed. Packt Publishing, 2016.

PERKINS, J. **Python 3 Text Processing with NLTK 3 Cookbook**. Packt Publishing, 2014.